SIMATS ENGINEERING

ASSESSMENT TOOL

COURSE NAME: Theory of Computation **COURSE CODE:** CSA13
COURSE OUTCOME COVERED CO2: Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems (BL:3)

Assessment Tool Weightage: 20%
QUESTIONS: 5
Total Marks: 25
Duration: 1 Hour

Simulation

Code of Conduct:
I S Venkata Abhishek - 192311053 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
S Venkata Abhishek

1. Design a DFA that accepts binary strings ending with "01". Minimize the DFA and explain its efficiency.

Given

Alphabet:
Σ = {0,1}

Language:
All binary strings that **end with "01"**.

DFA Design

States

q0 – Initial state

q1 – Last symbol is 0

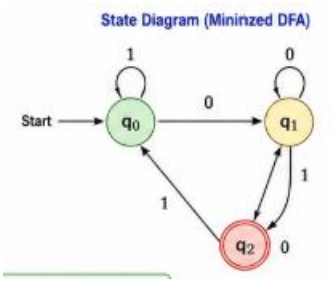
q2 – String ends with 01 (**Final State**)

Transition Table

Present State	Input 0	Input 1
→ q0	q1	q0
q1	q1	q2
*q2	q1	q0

(*Final State)

State Diagram



Minimized DFA

The DFA already contains only **3 distinguishable states**, so **no further minimization is possible**.

Efficiency Analysis

The minimized DFA:

Uses only three states.Requires less hardware memory.Performs constant-time validation.Suitable for ATM PIN verification, embedded systems, digital locks, and authentication modules.Reduces power consumption and increases processing speed.The minimized DFA efficiently validates all binary passwords ending with **01** while using the minimum number of states, making it ideal for hardware implementation.

2. Design a Regular Expression for "aaaabb". Explain the role of Regular Expressions in Web Applications.

Regular Expression:a^4b^2

aaaabb

Applications in Web Applications

Regular expressions are widely used to validate user inputs before data is processed.

Examples include:

- Email validation
- Password validation
- Phone number validation
- PIN verification
- Username validation
- ZIP code validation
- URL validation

Advantages

- Fast input checking
- Prevents invalid data entry
- Improves website security
- Reduces server processing
- Enhances application performance

Real-world Example

Password Rule:Password must contain exactly four 'a' followed by two 'b'.

Regex:a^4b^2

Accepted:aaaabb

Rejected:aaabb,aaaab,aaaabbb

Regular expressions provide a simple and efficient mechanism for validating user input, ensuring correctness and improving the security of web applications.

3. Convert Regular Expression a*b* into an Equivalent Finite Automaton. Explain their Equivalence.

Regular Expression

a*b*

Meaning:Zero or more a's ,Followed by zero or more b's

Examples Accepted:

ε
a
aa
aaa
b
bb
ab
aab
aaabbb

Rejected:

aba
bab
abba
baa

Equivalent Finite Automaton

States:

q0 – Start

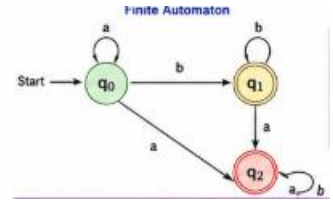
q1 – Reading b's

q2 – Dead state

Transition Table

State	a	b
→ q0	q0	q1
*q1	q2	q1
q2	q2	q2

Finite Automaton Diagram



Why are Regular Expressions and Finite Automata Equivalent?

Both recognize exactly the **regular languages**.

Every regular expression can be converted into:NFA,DFA

Similarly,Every DFA or NFA can be represented as a regular expression.

Therefore,Regular Expression ⇔ Finite Automaton

Applications

- Intrusion Detection Systems
- Lexical Analysis in Compilers
- Search Engines
- Log Monitoring
- Spam Filters
- Pattern Matching

The expression **a***b* and its finite automaton recognize the same language, demonstrating the equivalence between regular expressions and finite automata.

4. Prove Using Pumping Lemma that L = {$a^n b^n$ | n ≥ 0} is Not Regular. Explain its Compiler Implications.

Language

L = {$a^n b^n$ | n ≥ 0}

Examples:

ε
ab
aabb
aaabbb
aaaabbbb

Pumping Lemma

Assume L is regular.Then there exists a pumping length **p**.

Choose w = $a^p b^p$

Where |w| ≥ p

According to the pumping lemma,

w = xyz

such that |xy| ≤ p |y| > 0

Since |xy| ≤ p

the substring **y** contains only **a's**.

Pump i = 2

Result: $xy^2z = a^{(p+k)}b^p$

Where k > 0

Now, Number of a's ≠ Number of b's

Hence, $xy^2z \notin L$

This contradicts the Pumping Lemma.

Therefore, L is NOT Regular.

Compiler Implications

Finite automata cannot recognize nested structures like:

- Balanced parentheses
- Matching braces
- Equal numbers of symbols
- Function nesting
- Block structures
- Compilers therefore use:Pushdown Automata (PDA),Context-Free Grammars (CFG),Parsing algorithms
- instead of finite automata.

The pumping lemma proves that L = {$a^n b^n$} is not regular. Modern compilers rely on PDAs and CFGs to recognize such context-free language constructs.

5. Explain Closure Properties of Regular Languages and Their Importance.

Closure Properties

Regular languages remain regular after applying the following operations:Union (∪),Intersection (∩),Complement (′),Concatenation,Kleene Star (*),Difference,Reversal

Example

Rule 1:Strings ending with 01

Rule 2:Strings beginning with 11

Combined Rule: $L = L_1 \cup L_2$

The resulting language is still regular.

Why Does It Remain Regular?

Since DFAs can be combined using product construction, the new language is also recognized by a finite automaton.

Hence,

Regular + Regular = Regular

under closure operations.

Applications

Closure properties are used in:

- Intrusion Detection Systems
- Spam Filters
- Firewalls
- Email Validation
- Compiler Token Recognition
- Network Packet Filtering
- Input Validation
- Search Engines

Advantages

- Easy combination of multiple rules.
- Efficient DFA implementation.
- Fast pattern matching.
- Low memory usage.
- High performance in real-time systems.

Closure properties allow multiple regular-language rules to be combined while ensuring that the resulting language remains regular. This guarantees efficient implementation using finite automata in real-world applications such as spam filtering, intrusion detection, and input validation.